

Objetivo, alcance y preparación

Control

Versión 2.0 | Corte documental: 29/09/2026. Documento técnico para revisión académica. Implementado significa observado en código; no equivale a validación productiva. Las iteraciones son planes de trabajo, no actas ejecutadas.

Objetivo

Diseñar cobro verificable sin alterar el núcleo hasta aprobar el modelo comercial.

Situación observada

No implementado; diseño propuesto. No hay suscripción ni pasarela configurada.

Dependencias

Decidir si se cobra visita, comisión o suscripción; proveedor y obligaciones operativas; I02/I08/I10.

Historias de usuario

ID	Historia
HU-14	Como cliente quiero conocer qué pago y recibir un estado verificable sin cargos duplicados.

Trazabilidad y antecedentes

RF-32 previo (método de pago), EX-PAY-01 (cobro real nuevo). A10, A15, A32. Las referencias [Axx] remiten a las fichas del documento 01 y al catálogo trazabilidad/v2/articulos.csv. Páginas citadas: numeración física del PDF, salvo indicación expresa. Los originales se conservan en fuentes/articulos/Axx.pdf; SHA-256 en fuentes/articulos/manifest.json.

Incluye

payment_intents, payment_events y refunds propuestos. Importe en unidades menores enteras, moneda, referencia única, estado; sin PAN/CVV. Tabla de suscripciones solo tras aprobación independiente.

No incluye / riesgo principal

RF-32 no autoriza por sí solo cobro integrado. La evidencia de correlación en A10 no justifica imponer comisiones ni suscripción.

Entrada a desarrollo

Product Owner aprueba alcance y criterio; equipo dispone de entorno QA, datos sintéticos y dependencias. Definir capacidad del sprint antes de comprometer fechas. Si esta iteración es propuesta, la documentación no significa aprobación de implementación.

Iteración 9: Pagos y operación productiva: extensión condicionada

Implementación por capas

Frontend

Proponer checkout alojado por proveedor; mostrar pendiente, pagado y fallido desde servidor. Redirección exitosa no prueba pago. No pedir tarjeta en formularios propios.

Backend Laravel

Proponer pagos, intentos y eventos únicos por proveedor. Verificar firma de webhook, monto, moneda y recurso; transacción idempotente. Conciliación y reembolso con auditoría. No activar una suscripción inexistente al volver de checkout.

Python

Matching no participa en cobros ni recibe datos financieros. No cambiar ranking para privilegiar pagos sin regla de producto explícita.

Base de datos y contrato

payment_intents, payment_events y refunds propuestos. Importe en unidades menores enteras, moneda, referencia única, estado; sin PAN/CVV. Tabla de suscripciones solo tras aprobación independiente.

Secuencia para desarrollar

1. Reproducir situación actual y anotar commit. 2. Escribir caso negativo o límite que falle. 3. Implementar cambio mínimo en capa responsable. 4. Verificar respuesta y persistencia. 5. Refactorizar sin cambiar contrato. 6. Ejecutar regresión y revisar con otro integrante. 7. Actualizar evidencia y desplegar QA.

Errores y seguridad

La API vuelve a validar permisos y datos aunque la UI bloquee una acción. No usar localStorage como control real. Mantener mensajes comprensibles, datos de prueba sintéticos y secretos fuera de capturas/logs. No registrar un cambio como completo si solo existe en el frontend.

Iteración 9: Pagos y operación productiva: extensión condicionada

Pruebas, entrega y finalización

Casos de aceptación

ID	Procedimiento	Resultado esperado
I09-01	Webhook repetido y fuera de orden	No duplicar cargo ni degradar pago confirmado
I09-02	Firma inválida o monto diferente	Rechazar, registrar incidente y no activar nada
I09-03	Proveedor timeout después de aceptar pago	Conciliar antes de reintentar cargo
I09-04	Cliente intenta consultar pago ajeno	403/404 consistente sin filtración

Ejecución TDD

RED: ejecutar el caso que demuestra la falta o fallo, conservar resultado. GREEN: implementar y repetir. REFACTOR: limpiar código y ejecutar regresión pertinente. Para funciones ya implementadas, registrar prueba de caracterización y no inventar una fase RED anterior.

Despliegue de la iteración

Sandbox de proveedor primero; documentación y secretos separados. Producción requiere decisión comercial, QA de seguridad y responsable de conciliación. No se realizan pagos reales en esta tarea.

Criterios de salida

Historias aceptadas, permisos negativos aprobados, contrato y diccionario actualizados, pruebas pertinentes sin fallos y QA verificable. Guardar build/commit, entorno, caso y observado. Riesgos pendientes con responsable por asignar; ninguna firma ni aceptación simulada.

Carpeta de evidencia

evidencias/v2/I09/ contiene README de estado. En la ejecución se añadirán logs, casos, incidentes, retest, acta de revisión y registro de despliegue. Actualmente el estado es PLANIFICADO / NO EJECUTADO como iteración. No confundir pruebas previas con cierre de sprint.